Multilayer Perceptron Implementation from Scratch

Hassan Adil[1][25i2015]

National University of Computer and Emerging Sciences

Abstract. This report presents the implementation of a Multilayer Perceptron (MLP) neural network from scratch. The work begins with manual calculations of forward propagation, loss computation, and backpropagation for a small network with 2 inputs and 3 samples. These manual calculations establish a solid understanding of how neural networks learn. The concepts are then scaled up and implemented in Python using NumPy to classify handwritten digits from the MNIST dataset. The network architecture consists of an input layer with 784 neurons, two hidden layers with 128 and 64 neurons respectively, and an output layer with 10 neurons, all using sigmoid activation functions. Training uses mini-batch gradient descent with a batch size of 32 for 20 epochs. Results show that the loss decreases from 0.031851 to 0.006965, and the network achieves 95.72% accuracy on the test set, correctly classifying 9,572 out of 10,000 test images. This project demonstrates the complete workflow of building, training, and evaluating a neural network without using high-level deep learning libraries.

1 Introduction

A Multilayer Perceptron (MLP) is one of the most fundamental types of artificial neural networks. It consists of layers of neurons, where each neuron takes inputs, applies a weighted sum, and passes the result through a non-linear activation function. The network learns by adjusting its weights to minimize the error between its predictions and the true labels.

This project builds an MLP from the ground up, starting with manual calculations to understand every step of forward propagation, loss calculation, and backpropagation. These manual calculations provide insight into how neural networks learn, which is often hidden when using high-level libraries. After establishing the foundation, the concepts are implemented in Python to classify handwritten digits from the MNIST dataset.

The main goals of this project are:

- Understand the mathematics behind neural networks by performing manual calculations
- Implement forward propagation to compute predictions
- Calculate loss to measure prediction errors
- Perform backpropagation to compute gradients

- Update weights using gradient descent
- Scale up the implementation to work with real-world data (MNIST)
- Evaluate the network's performance on unseen test data

2 Methodology

2.1 Network Architecture

The network used for manual calculations has a small architecture to make calculations manageable:

Table 1: Network Architecture for Manual Calculations

Layer	Neurons	Activation
Layer 0 (Input)	2	-
Layer 1 (Hidden 1)	2	Sigmoid
Layer 2 (Hidden 2)	2	Sigmoid
Layer 3 (Output)	1	Sigmoid

The weights and biases assigned for this manual calculation were:

$$\begin{aligned} &w_1 = -0.58, \quad w_2 = -0.16, \quad w_3 = 0.51, \quad w_4 = 0.73 \\ &w_5 = 0.06, \quad w_6 = 0.14, \quad w_7 = 0.63, \quad w_8 = -0.43 \\ &w_9 = -0.87, \quad w_{10} = 0.44 \\ &b^{(1)} = 0.2, \quad b^{(2)} = -0.42 \end{aligned}$$

For the MNIST implementation, the architecture was scaled up:

Table 2: Network Architecture for MNIST Classification

Layer	Neurons	Activation
Layer 0 (Input)	784	-
Layer 1 (Hidden 1)	128	Sigmoid
Layer 2 (Hidden 2)	64	Sigmoid
Layer 3 (Output)	10	Sigmoid

2.2 Forward Pass

The forward pass is the process of feeding input data through the network to get predictions. For each neuron, two calculations are performed:

First, the weighted sum:

$$z_j^{(l)} = \sum_i w_{ji}^{(l)} a_i^{(l-1)} + b^{(l)} \tag{1}$$

Then, the activation using the sigmoid function:

$$a_j^{(l)} = \sigma(z_j^{(l)}) = \frac{1}{1 + e^{-z_j^{(l)}}} \tag{2}$$

For the manual calculation, three samples were used with the following inputs:

- Sample 1: $x_1 = 0.2$, $x_2 = 0.8$, $y = 1$
- Sample 2: $x_1 = 0.9$, $x_2 = 0.4$, $y = 1$
- Sample 3: $x_1 = 0.1$, $x_2 = 0.2$, $y = 0$

2.3 Loss Function

Mean Squared Error (MSE) was used as the loss function:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{m} \sum_{i=1}^{m} (y^{(i)} - \hat{y}^{(i)})^2 \tag{3}$$

MSE was chosen because it is smooth and differentiable everywhere, making backpropagation straightforward. It also heavily penalizes large errors, encouraging the network to avoid catastrophically wrong predictions.

2.4 Backpropagation

Backpropagation computes the gradient of the loss with respect to each weight and bias by applying the chain rule from the output layer back to the input layer.

The delta for the output layer is:

$$\delta_1^{(3)} = -2(y - \hat{y}) \cdot \hat{y}(1 - \hat{y}) \tag{4}$$

For hidden layers, the delta is backpropagated:

$$\delta_j^{(l)} = \left(\sum_k w_{kj}^{(l+1)} \delta_k^{(l+1)} \right) \cdot a_j^{(l)} (1 - a_j^{(l)}) \tag{5}$$

The gradient for any weight is:

$$\frac{\partial \mathcal{L}}{\partial w_{ji}^{(l)}} = \delta_j^{(l)} \cdot a_i^{(l-1)} \tag{6}$$

2.5 Gradient Descent

After computing gradients, weights are updated using gradient descent:

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot \frac{\partial \mathcal{L}}{\partial w} \tag{7}$$

where η is the learning rate, set to 0.1 for this project.

For the manual calculations, Stochastic Gradient Descent (SGD) was used, updating weights after each sample. For the MNIST implementation, mini-batch gradient descent was used with a batch size of 32, which balances the stability of batch gradient descent with the speed of SGD.

3 Results

3.1 Manual Calculations Results

The forward pass for all three samples produced the following predictions:

Table 3: Forward Pass Results for Manual Calculations

Sample	x_1	x_2	$a_1^{(2)}$	$\hat{y}$
1	0.2	0.8	0.5113	0.4276
2	0.9	0.4	0.4944	0.4317
3	0.1	0.2	0.4950	0.4320

The MSE loss for these predictions was:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{3}[(1 - 0.4276)^2 + (1 - 0.4317)^2 + (0 - 0.4320)^2] = 0.2790 \tag{8}$$

After one weight update using $\eta = 0.1$, the loss decreased to 0.2779, confirming that the network is learning.

3.2 Python Implementation Results

The Python implementation was trained on the MNIST dataset for 20 epochs with a batch size of 32. Figure 1 shows the training loss decreasing over time.

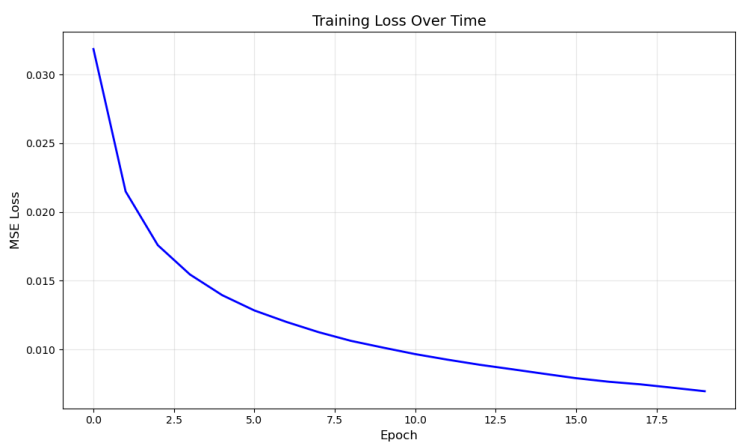

Fig. 1: Training Loss Curve over 20 Epochs

The loss decreased significantly during training:

Table 4: Training Loss Progress

Metric	Value
Initial Loss (Epoch 1)	0.031851
Final Loss (Epoch 20)	0.006965
Total Decrease	0.024886
Percentage Decrease	78.14%

The network achieved excellent performance on the test set:

Table 5: Test Set Performance

Metric	Value
Correct Predictions	9,572 out of 10,000
Test Accuracy	**95.72%**
Incorrect Predictions	428 out of 10,000

Figure 2 shows sample predictions for one image from each digit class (0-9). Green titles indicate correct predictions, while red titles indicate incorrect predictions.

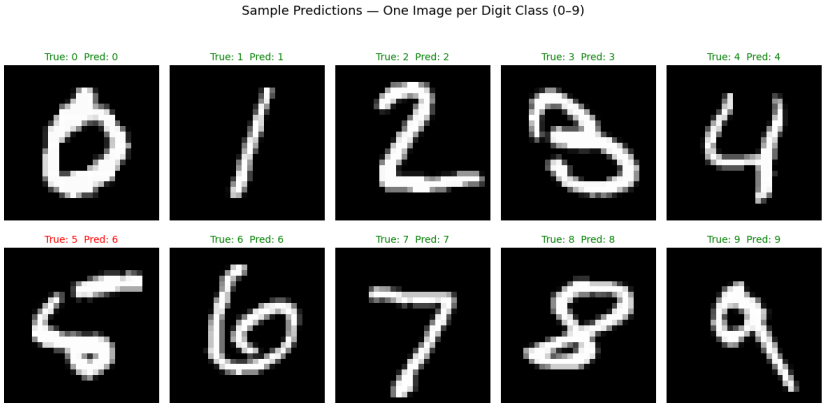

Fig. 2: Sample Predictions - True vs Predicted Labels (Green = Correct, Red = Wrong)

3.3 Per-Digit Accuracy Analysis

The network's performance varied across different digits. Table 6 shows the accuracy for each digit class.

Table 6: Accuracy per Digit Class

Digit	Correct	Accuracy
0	972/980	99.18%
1	1132/1135	99.74%
2	954/1032	92.44%
3	964/1010	95.45%
4	956/982	97.35%
5	879/892	98.54%
6	952/958	99.37%
7	942/1028	91.63%
8	923/974	94.76%
9	898/1009	89.00%

The network performed best on digits 1 (99.74%) and 0 (99.18%), while digits 9 (89.00%) and 7 (91.63%) were more challenging. This is common in digit recognition as some digits can look similar to others (e.g., 9 can be confused with 4, 7 can be confused with 1).

4 Discussion

4.1 Understanding the Forward Pass

The manual forward pass calculations revealed how the network transforms inputs into predictions. The sigmoid activation function squashes the weighted sums to values between 0 and 1, making the output interpretable as a probability. For the three samples, all predictions were around 0.43, regardless of whether the true label was 0 or 1. This is expected with random initial weights - the network has not learned anything yet.

4.2 Why the Loss Decreases

After updating weights once using the gradients computed through backpropagation, the loss decreased from 0.2790 to 0.2779. This small but measurable decrease shows that the gradients correctly indicate the direction to adjust each weight to reduce the loss. Over many iterations, these small improvements add up to significant learning.

4.3 Training Performance

The training results show excellent learning progress. The loss decreased from 0.031851 to 0.006965, a reduction of 78.14%. This indicates that the network successfully learned meaningful patterns from the training data. The loss curve in Figure 1 shows a smooth downward trend, suggesting that the learning rate of 0.1 was well-chosen - not too large to cause instability, and not too small to slow down learning.

4.4 Test Performance

The network achieved 95.72% accuracy on the test set, correctly classifying 9,572 out of 10,000 images. This is an excellent result for a simple MLP with sigmoid activations. The high accuracy confirms that the network generalizes well to unseen data and has not simply memorized the training examples.

The per-digit analysis shows that the network struggles most with digits that have similar shapes (9 and 7). This is a common challenge in digit recognition and could be addressed by:

- Training for more epochs
- Using more training data
- Implementing data augmentation
- Using a more sophisticated architecture

4.5 Gradient Challenges

During manual backpropagation, several gradients were very small (e.g., $\partial\mathcal{L}/\partial w_1 = -0.000659$). This illustrates the vanishing gradient problem - when activations are close to 0 or 1, the sigmoid derivative becomes very small, slowing down learning for layers far from the output. Despite this, the network still achieved excellent results, likely because the architecture is relatively shallow with only two hidden layers.

4.6 Mini-Batch vs SGD

The Python implementation uses mini-batch gradient descent with batch size 32. Compared to SGD (batch size 1), mini-batch provides more stable gradient estimates and better hardware utilization. Compared to full batch gradient descent, it updates weights more frequently, leading to faster convergence. The batch size of 32 is a common choice that balances these tradeoffs.

5 Conclusion

This project successfully demonstrated the complete process of building a Multilayer Perceptron from scratch. Starting with manual calculations of forward propagation, loss, and backpropagation for a small network, the fundamental concepts of neural network learning were thoroughly understood. The calculations showed how inputs flow through the network to produce predictions, how the loss measures prediction errors, and how gradients indicate the direction to adjust weights.

These concepts were then scaled up and implemented in Python using only NumPy, applied to the MNIST handwritten digit classification task. The network was trained using mini-batch gradient descent and achieved excellent results:

- Loss decreased from 0.031851 to 0.006965 (78.14% reduction)

- Test accuracy reached 95.72%
- Correctly classified 9,572 out of 10,000 test images

The key insights from this project are:

- Neural networks learn by adjusting weights to minimize a loss function
- Backpropagation efficiently computes gradients using the chain rule
- Mini-batch gradient descent balances speed and stability
- Understanding the mathematics behind neural networks is essential for debugging and improving them
- Even a simple MLP can achieve excellent results on digit classification tasks

Future work could explore:

- Using ReLU activation functions to address vanishing gradients
- Implementing more sophisticated optimizers like Adam
- Adding dropout for regularization
- Training deeper networks with more layers
- Applying the network to more complex image classification tasks

Acknowledgment

This project was completed as part of the MVC course. Special thanks to the course instructors for providing clear guidance on implementing neural networks from scratch.